

Project 2 Pitch: Predicting 30-Day Readmission for Diabetic Inpatients

Data Science Capstone — End-to-End ML Pipeline

Dataset	Diabetes 130-US Hospitals (1999–2008), UCI ML Repository
Records	101,766 inpatient encounters across 130 US hospitals
Problem Type	Binary classification — readmitted within 30 days (yes / no)
Methodology	CRISP-DM lifecycle with scikit-learn Pipeline
Tools	Python • Pandas • scikit-learn • XGBoost • SHAP • Matplotlib/Seaborn

1. Business Problem

1.1 The Problem

Roughly 20% of Medicare patients are readmitted to a hospital within 30 days of discharge. Under the Hospital Readmissions Reduction Program (HRRP), CMS penalizes hospitals with excess 30-day readmission rates by reducing Medicare payments up to 3% across all inpatient claims — not just the readmitted cases. In FY 2024, 9 hospitals received the maximum 3% penalty, and roughly 80% of evaluated hospitals incurred some level of penalty, costing the industry over \$560 million annually.

For diabetic patients specifically, the challenge is compounded. Diabetes management in hospital settings is often inconsistent, glycemic control is frequently suboptimal during acute admissions, and discharge planning for patients with complex medication regimens (insulin, metformin, multiple oral agents) requires coordination that hospitals often fail to deliver. The result: diabetic patients who don't receive appropriate in-hospital management return at higher rates, driving both financial penalties and adverse clinical outcomes.

The core business question is: given the clinical, demographic, and medication data available at the time of discharge, can we identify which diabetic patients are most likely to be readmitted within 30 days? If so, hospitals can target transitional care resources — discharge coaching, medication reconciliation, follow-up scheduling — at the patients who need them most, rather than applying expensive interventions uniformly.

1.2 Stakeholders

Stakeholder	How They Use the Model
Chief Medical Officer (CMO)	Sets clinical policy on discharge protocols. Uses risk scores to justify investment in transitional care programs and to benchmark performance against peer hospitals.
VP of Case Management / Utilization Management	The primary operational consumer. Uses patient-level risk flags to prioritize which discharges receive intensive follow-up (e.g., pharmacist-led medication reconciliation, 48-hour post-discharge call, home health referral).
Chief Financial Officer (CFO)	Quantifies the financial exposure from HRRP penalties. Uses the model's predicted readmission volume to estimate ROI on intervention programs: cost of intervention vs. penalty savings.
Hospitalists / Endocrinologists	Frontline clinicians who receive risk alerts at discharge. Feature importance (e.g., HbA1c not measured, high number of inpatient medications) guides specific clinical actions.
Quality / Population Health Team	Monitors aggregate readmission trends by patient segment, tracks intervention effectiveness over time, and reports to CMS.

1.3 Why Machine Learning

Simple rule-based approaches (e.g., “flag all patients over 65 with more than 5 medications”) cannot capture the non-linear interactions between clinical variables that drive readmission. A patient’s readmission risk depends on the interplay of diagnosis complexity, medication changes during the stay, whether HbA1c was measured, length of stay, number of prior visits, discharge disposition, and dozens of other factors. Machine learning models — particularly gradient-boosted ensembles — excel at learning these multi-variable interaction patterns from historical data without requiring manual rule specification.

Additionally, the dataset’s class imbalance (roughly 11% readmitted within 30 days vs. 89% not) and its mix of continuous, ordinal, and high-cardinality categorical variables make it a natural fit for ML pipelines with proper preprocessing, rather than traditional statistical models.

1.4 Dataset Description

Source: UCI Machine Learning Repository (ID 296), originally published by Strack et al. (2014) in *BioMed Research International*. Licensed under CC BY 4.0.

Size: 101,766 encounters × 50 features. Each row is one inpatient encounter for a patient diagnosed with diabetes who stayed 1–14 days and had laboratory work and medications administered.

Key variable groups:

- **Demographics:** race, gender, age (bucketed in 10-year ranges), weight (mostly missing)
- **Encounter details:** admission_type_id, discharge_disposition_id, admission_source_id, time_in_hospital, number_inpatient (prior inpatient visits in past year), number_outpatient, number_emergency
- **Clinical:** diag_1/2/3 (primary, secondary, tertiary ICD-9 codes — ~700 unique values each), number_diagnoses, max_glu_serum, A1Cresult
- **Medications:** 23 medication columns (metformin, insulin, glipizide, etc.) with values “No” / “Steady” / “Up” / “Down”, plus num_medications, diabetesMed (yes/no), change (medication changed during stay)
- **Target variable:** readmitted — original values are “<30” (readmitted within 30 days), “>30” (readmitted after 30 days), “NO” (not readmitted). I will binarize to <30 = 1, else = 0.

1.5 Success Metrics

Metric	Technical Definition	Business Relevance
ROC-AUC	Area under receiver operating characteristic curve. Measures discrimination across all thresholds.	Primary model comparison metric. Target: >0.65 (published benchmark for this dataset is 0.64–0.67).
Recall (Sensitivity)	Proportion of actual readmissions correctly identified.	Critical for the business case: a missed readmission = a penalty. High recall is more valuable than high precision.
Precision	Proportion of flagged patients who actually readmit.	Controls intervention cost. If precision is too low, the hospital wastes resources on patients who would not have readmitted.
F1-Score	Harmonic mean of precision and recall.	Balances the cost of missed readmissions vs. wasted interventions.
Cost-Adjusted Net Benefit	Custom metric: (True Positives × penalty-saved) – (False Positives × intervention-cost).	Translates model performance into dollar-denominated ROI. Answers: does deploying this model save more than it costs?

2. Problem-Solving Process

2.1 Data Acquisition and Understanding

Acquisition: Download from UCI via the ucimlrepo Python package (`fetch_ucirepo(id=296)`) or directly from Kaggle. Single CSV file, no API authentication required.

Initial exploration plan:

- Profile every column: dtype, unique count, null count, null percentage
- Examine the target distribution — expect heavy class imbalance (~11% readmitted <30 days)
- Identify columns with excessive missing data (weight is ~97% missing and will be dropped)
- Check for duplicate encounters per patient_nbr — the same patient may have multiple encounters, which risks data leakage if not handled
- Inspect the ICD-9 diagnosis codes (diag_1/2/3) — ~700 unique values each, requiring grouping into clinical categories

Data quality concerns to investigate:

- Encounters where discharge_disposition_id indicates “Expired” (patient died) — these must be excluded since a deceased patient cannot be readmitted
- The “gender” column contains an “Unknown/Invalid” category
- The IDs for admission_type, discharge_disposition, and admission_source are numeric codes that map to categories in a separate IDs_mapping.csv file

2.2 Data Preparation and Feature Engineering

Cleaning Steps

1. Binarize target: “<30” → 1, everything else → 0
2. Remove expired patients: drop rows where discharge_disposition_id maps to “Expired” (codes 11, 19, 20, 21)
3. Deduplicate by patient: keep one encounter per patient_nbr (the encounter with the longest time_in_hospital) to avoid data leakage from correlated encounters
4. Drop columns with >50% missing: weight (~97% null) and payer_code (~40% null, and not clinically relevant to the prediction task)
5. Handle “?” values: the dataset uses “?” instead of NaN for missing categoricals. Replace with NaN, then impute or create an “Unknown” category as appropriate
6. Collapse rare categories: medical_specialty has ~70 unique values, many with <100 encounters. Group rare specialties into “Other”
7. Group ICD-9 diagnosis codes: map diag_1, diag_2, diag_3 from ~700 raw codes into 9 clinical categories (Circulatory, Respiratory, Diabetes, Digestive, Injury, Musculoskeletal, Genitourinary, Neoplasm, Other) following the Strack et al. (2014) methodology

Feature Engineering

- **medication_change_count:** Count of how many of the 23 medication columns show “Up” or “Down” (not “Steady” or “No”). Captures the intensity of medication adjustment during the stay.
- **total_visits_prior_year:** Sum of number_inpatient + number_outpatient + number_emergency. A composite utilization index.
- **has_A1c_measured:** Binary flag: 1 if A1Cresult is not “None”. The Strack et al. paper found that HbA1c measurement was independently associated with reduced readmission. This is a key feature.
- **A1c_elevated:** 1 if A1Cresult > 7 (poorly controlled diabetes).
- **diagnosis_is_diabetes:** 1 if the primary diagnosis (diag_1) is in the diabetes ICD-9 range (250.xx).
- **num_procedures_per_day:** num_procedures / time_in_hospital. Measures treatment intensity.
- **polypharmacy:** 1 if num_medications > 15. A recognized risk factor for adverse events and readmission.

scikit-learn Pipeline Design

The preprocessing pipeline will use ColumnTransformer to handle different feature types in parallel:

- Numeric features → SimpleImputer(strategy='median') → StandardScaler
- Low-cardinality categoricals (≤15 categories) → SimpleImputer(strategy='most_frequent') → OneHotEncoder(handle_unknown='ignore')
- High-cardinality categoricals (diag_1/2/3 grouped, medical_specialty) → OrdinalEncoder or TargetEncoder depending on model requirements
- Binary flags → passthrough

The full Pipeline chains the ColumnTransformer with SMOTE (or class_weight='balanced') and the classifier, so the entire flow from raw features to predictions is a single reproducible object.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer

preprocessor = ColumnTransformer([
    ('num', num_pipeline, numeric_features),
    ('cat', cat_pipeline, categorical_features),
    ('passthrough', 'passthrough', binary_features),
])

model_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', classifier), # Swap classifier during comparison
])
```

2.3 Modeling Strategy

Algorithms to Evaluate (Minimum 3)

Model	Why Include It	Expected Strengths
Logistic Regression	Interpretable baseline. Coefficients translate to odds ratios that clinicians understand.	Fast training. Clear feature importance. Regularization (L1/L2) handles correlated features.
Random Forest	Captures non-linear interactions without manual feature crosses. Robust to outliers.	Built-in feature importance. Handles mixed types natively. Less prone to overfitting than single trees.
XGBoost	State-of-the-art for tabular classification. The published benchmark for this dataset (AUC 0.667) used XGBoost.	Handles class imbalance via <code>scale_pos_weight</code> . Built-in regularization. SHAP integration for interpretability.
Support Vector Machine (SVM)	Different learning paradigm (margin-based). Provides diversity in the model comparison.	Effective in high-dimensional spaces after one-hot encoding. Kernel trick for non-linearity.

Cross-Validation and Tuning

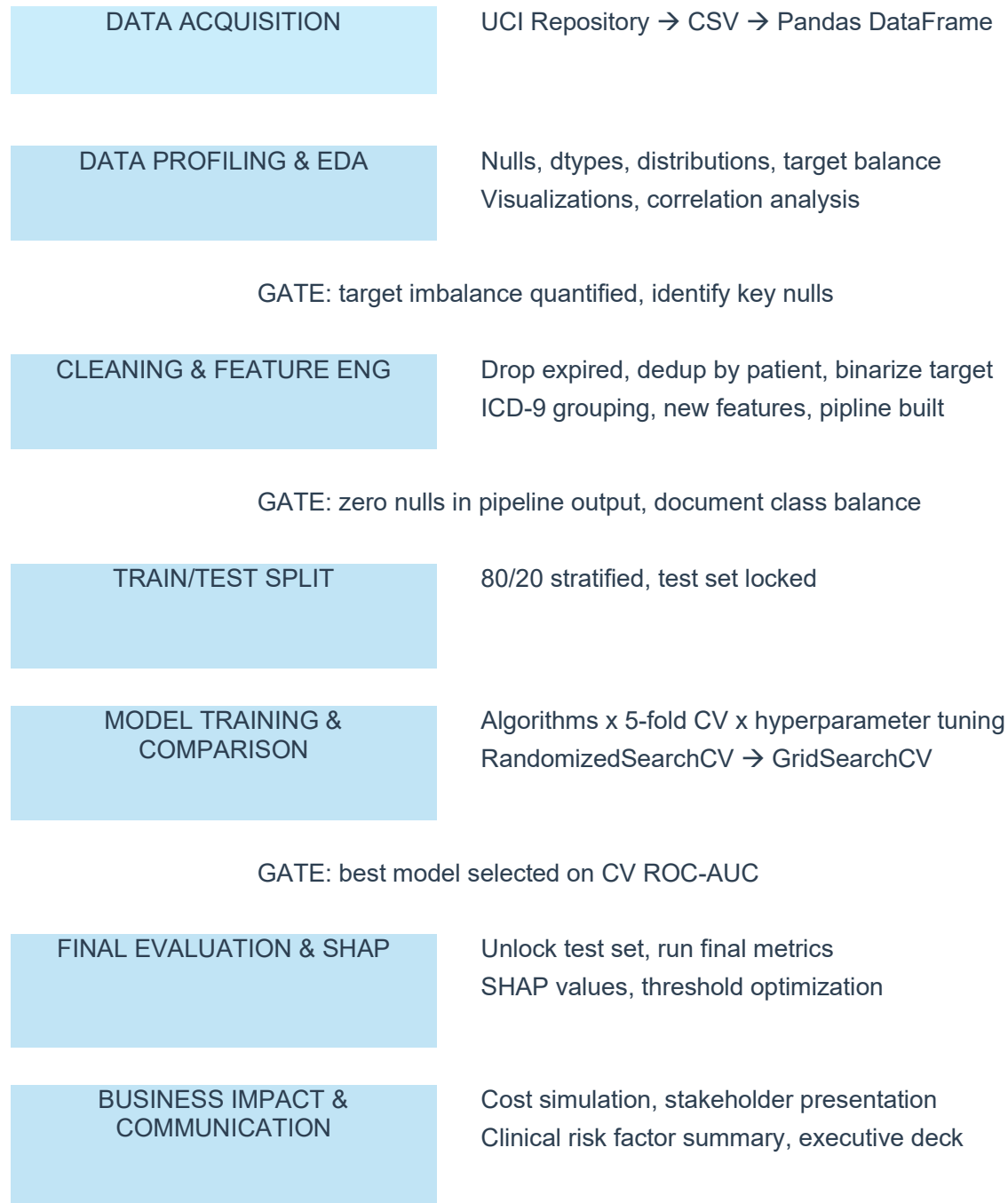
- **Split strategy:** 80/20 train-test split, stratified on the target. Within training, 5-fold stratified cross-validation for all hyperparameter tuning. The test set is held out until final evaluation.
- **Class imbalance handling:** Evaluate three approaches: (1) `class_weight='balanced'`, (2) SMOTE oversampling within CV folds (using `imblearn Pipeline` to prevent leakage), (3) `scale_pos_weight` for XGBoost. Compare which yields the best recall without destroying precision.
- **Hyperparameter search:** `RandomizedSearchCV` for initial broad sweep (50–100 iterations), followed by `GridSearchCV` on a narrowed parameter space around the best region. Scoring metric: ROC-AUC.
- **Key hyperparameters:** LR: `C`, `penalty` (L1/L2). RF: `n_estimators`, `max_depth`, `min_samples_leaf`. XGBoost: `learning_rate`, `max_depth`, `n_estimators`, `subsample`, `colsample_bytree`, `scale_pos_weight`. SVM: `C`, `kernel`, `gamma`.

2.4 Results Interpretation and Communication

- SHAP summary plots for the final model — global feature importance plus individual patient-level explanations
- Confusion matrix heatmaps for each model at the chosen operating threshold
- ROC curves overlaid for all four models on a single plot
- Precision-recall curves (more informative than ROC for imbalanced datasets)
- A “business impact simulation”: for a hypothetical hospital with 10,000 Medicare discharges, calculate the number of readmissions correctly flagged, the number of unnecessary interventions, and the net financial impact at different probability thresholds
- A non-technical “risk factor” summary: translate the top 10 SHAP features into plain-English clinical insights (e.g., “Patients whose HbA1c was not measured during the stay were 1.4× more likely to be readmitted”)

2.5 Conceptual Framework — Solution Pipeline Flowchart

The following flowchart outlines the end-to-end pipeline from raw data to business recommendations. Each stage has explicit dependencies on the preceding stage, and validation gates prevent proceeding on bad data.



Key dependencies: Stage 3 (cleaning) must complete before Stage 4 (splitting) to prevent data leakage from deduplication. SMOTE is applied only within CV folds in Stage 5, never on the test set. SHAP analysis in Stage 6 requires the final trained model from Stage 5.

3. Timeline and Scope

The project spans 3 weeks. Each phase has a defined deliverable and a checkpoint before proceeding.

Week	Phase	Deliverables	Checkpoint
1	Problem Definition & Data Acquisition	Pitch document submitted. Dataset downloaded, loaded into Pandas. Project repo initialized with README, .gitignore, and directory structure. Initial profiling notebook with null counts, dtypes, target distribution.	Dataset loads cleanly. Target distribution confirmed (~11% readmit <30).
1	Exploratory Data Analysis	Comprehensive EDA notebook section: target distribution, feature distributions (numeric histograms, categorical bar charts), correlation analysis, ICD-9 code distribution analysis, medication pattern analysis. Identify top features by visual inspection.	All columns profiled. Key hypotheses documented (e.g., A1c measurement, prior utilization).
1	Data Preprocessing & Pipeline	Full cleaning pipeline: dedup by patient, remove expired, binarize target, ICD-9 grouping, feature engineering (7 new features). sklearn ColumnTransformer + Pipeline assembled and tested. Train/test split created.	Pipeline transforms raw data to model-ready matrix without errors. Test set locked.
2	Model Development	Train baseline models (LR, RF, XGBoost, SVM) with default hyperparameters. Compare 5-fold CV scores. Begin hyperparameter tuning with RandomizedSearchCV. Document iterative improvement.	Baseline CV scores for all 4 models logged. Best model AUC > random (0.5).
2	Model Evaluation & Refinement	Final hyperparameter tuning (GridSearchCV). Select best model. Unlock test set and evaluate. SHAP analysis. Precision-recall threshold optimization. MVP notebook ready for peer review.	Final model selected. Test-set metrics computed. SHAP plots generated.
2	Documentation & Business Impact	Code cleanup and commenting. Business impact simulation (cost/benefit at multiple thresholds). Executive presentation slides. Incorporate peer feedback from MVP review.	Notebook runs top-to-bottom. Presentation ready for recording.
3	Final Review & Submission	Quality assurance pass. Record presentation video. Final GitHub push. Submit.	Video recorded. Repo clean. Submitted.

3.1 Potential Challenges and Additional Research

Challenge	Mitigation	Research Needed
Class imbalance (~11% positive class)	Compare SMOTE, class_weight, and threshold tuning. Prioritize recall over accuracy.	Read imblearn documentation on SMOTE variants (SMOTE-ENN, BorderlineSMOTE).
High-cardinality ICD-9 codes (~700 levels per column)	Group into 9 clinical categories per Strack et al. methodology.	Study the ICD-9 code ranges and the original paper's Table 2 mapping.
Multiple encounters per patient	Keep only one encounter per patient to avoid leakage from correlated outcomes.	Review literature on handling repeated measures in classification tasks.
Dataset age (1999–2008)	Acknowledge as a limitation. Note that clinical patterns of readmission are well-established and the HRRP penalty structure is current.	Review recent HRRP penalty data to confirm business case is still active.
Modest expected AUC (0.65–0.67 benchmark)	Frame as realistic for readmission prediction (social determinants not in data). Focus the story on feature insights and business impact, not raw accuracy.	Study SHAP documentation for clinical-interpretability best practices.
Translating model into business value	Build a cost-benefit simulation: penalty per readmission vs. cost of transitional care intervention per flagged patient.	Research CMS HRRP penalty calculation methodology and average hospital penalty amounts.

Note on Dataset Relevance

While the dataset spans 1999–2008, the business problem it addresses is more relevant today than when the data was collected. The HRRP did not take effect until 2013, and penalties have been assessed every year since. In FY 2024, over 2,500 hospitals were penalized, with total industry penalties exceeding \$560 million. The clinical predictors of diabetic readmission (medication management, glycemic control, discharge disposition, utilization history) have not fundamentally changed, making this dataset appropriate for developing and benchmarking a readmission prediction pipeline even if the specific model would need retraining on current EHR data before clinical deployment.